Constructing Optimal Rank-Decomposition Trees of Halin Graphs

Sheng-Lung Peng* Guan-Da Chen Hsiang-Yu Tsou In-Te Li Yan-Fang Li

Department of Computer Science and Information Engineering
National Dong Hwa University, Hualien 974, Taiwan

Abstract

In this paper, we show that the rankwidth of a Halin graph G is at most three and propose a linear-time algorithm to construct an optimal rank-decomposition tree for G.

1 Introduction

Let $G = (V, E)$ be a simple, finite, and undirected graph. Let $n = |V|$ and $m = |E|$. Robertson and Seymour introduced the concept of treewidth on graphs [11, 12]. Further, it has been shown that many NP-hard graph problems can be solved in polynomial time on graphs of bounded treewidth.

However, large cliques are excluded by graphs of bounded treewidth. Courcelle and Olariu introduced the concept of cliquewidth to exclude the weakness of treewidth [2]. That is, a graph with bounded cliquewidth can contain large cliques, *e.g.*, the cliquewidth of any clique is one.

Similar to the treewidth, many NP-hard graph problems can be solved in polynomial time on graphs of bounded cliquewidth. Unfortunately, Fellows, *et al.* [4, 5] have shown that the problem of deciding whether a graph has cliquewidth at most k is NP-complete if k is given as an input.

Instead, Oum and Seymour proposed the concept of rankwidth [9]. A *cubic* (respectively, *subcubic*) tree is a tree such that every internal vertex has degree 3 (respectively, at most 3). A *rank-decomposition* of a graph G is a pair $(\mathcal{T}, \mathcal{L})$, where $\mathcal{T}$ is a cubic tree and $\mathcal{L} : V \to \{v \mid v \text{ is a leaf of } \mathcal{T}\}$ is a bijective function. In the case that $|V| = 2$, $\mathcal{T}$ contains only one edge. However, there is no rank-decomposition of G if $|V| = 1$. For an edge e of $\mathcal{T}$, the connected components of $\mathcal{T} - e$ induce a partition $\{X, Y\}$ of the set of leaves of $\mathcal{T}$. Since $\mathcal{L}$ is bijection, (X, Y) is also a bipartition of V. Let $X = \{u_1, \ldots, u_r\}$ and $Y = \{v_1, \ldots, v_s\}$. Let M_e be an $|X| \times |Y|$ 0-1 matrix such that $M_e[i, j] = 1$ if $(u_i, v_j) \in E$; otherwise, $M_e[i, j] = 0$. The *width* of edge e of a rank-decomposition $(\mathcal{T}, \mathcal{L})$ is the rank of the matrix M_e. The *width* of $(\mathcal{T}, \mathcal{L})$ is the maximum width of M_e for all possible edges e of $\mathcal{T}$. The *rankwidth* $rwd(G)$ of G is the minimum width of all possible rank-decompositions of G. In the case of $|V| = 1$, $rwd(G) = 0$. Note that $\mathcal{T}$ is also called a *rank-decomposition tree* of G.

Let $twd(G)$ (respectively, $cwd(G)$) denote the treewidth (respectively, clique-width) of G. It is shown that $rwd(G) \leq cwd(G) \leq 2^{rwd(G)+1} - 1$ for any graph G [9]. It implies that $cwd(G)$ can be approximated by $rwd(G)$. That is, cliquewidth is equivalent to rankwidth in the sense that $cwd(G)$ is bounded. Let k be fixed. There is an $O(n^3)$-time algorithm that either confirms that an n-vertex input graph has rankwidth greater than k or outputs a rank-decomposition of width at most $3k + 1$ [8]. It is also known that $rwd(G) \leq twd(G) + 1$. Thus it is interesting to know whether optimal rank-decompositions of the graph classes of bounded treewidth can be constructed in time lesser than $O(n^3)$. Peng *et al.* show that the rankwidth of an outerplanar graph is at most two and propose a linear-time algorithm to construct its rank-decomposition tree [10]. In this paper we study the same problem on Halin graphs. We show that the rankwidth of a Halin graph is at most three and propose a linear-time algorithm to construct its rank-decomposition tree.

2 Halin graphs

Halin graphs is defined by Halin [14]. Let $T = (V, E')$ be a tree with no degree-2 vertex. A graph $G = (V, E)$ is *Halin* if G can be obtained from T by connecting its leaves into a cycle C that crosses none of its edges. Thus, Halin graphs are planar.

*Corresponding author: slpeng@mail.ndhu.edu.tw

The cycle C in G formed by the leaves of T is called the *outer cycle* of G. For convenience, we let $G = T + C$. Vertices in C are called *outer vertices*. Vertices in $T \setminus C$ are called *inner vertices*. Figure 1 shows an example of Halin graphs.

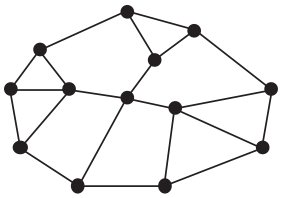


Figure 1: A Halin graph with 9 outer vertices and 4 inner vertices.

2.1 The rankwidth of Halin graphs

Note that the treewidth of a Halin graph is three. Therefore, the rankwidth of a Halin graph is at most four. In this section, we will show that the rankwidth of a Halin graph is at most three. First, it is not hard to check that the rankwidth of a graph with at most six vertices is at most two. Figure 2 shows this case.

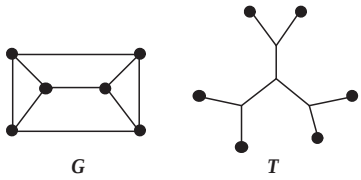


Figure 2: A 6-vertex graph G with a rank-decomposition tree $\mathcal{T}$ of rankwidth 2.

A graph G is *distance hereditary* if the distance of any two vertices remains the same in every induced connected subgraph of G [1]. Oum proved that the graphs of rankwidth 1 are exactly the distance-hereditary graphs [7]. Thus, if a Halin graph G is also distance hereditary, then its rankwidth is 1. If G is not distance hereditary and has at most six vertices, then its rankwidth is 2. For the other cases, only three special graphs depicted in Figure 3 are rankwidth-2 graphs.

In the following, we only consider Halin graphs with at least 7 vertices and we exclude the three special graphs depicted in Figure 3.

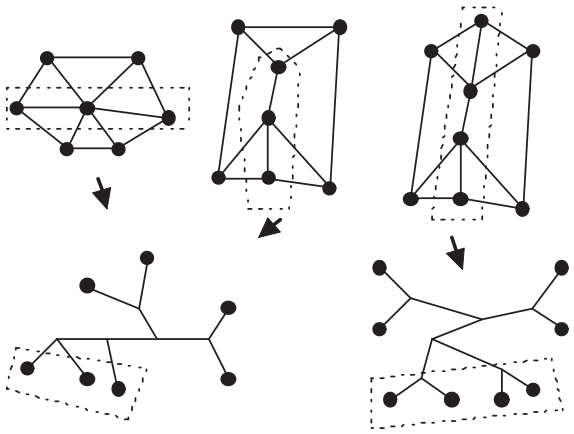


Figure 3: Three special Halin graphs with $|V| \geq 7$ and their rankwidth-2 decomposition trees.

For a Halin graph $G = (V, E)$, let $\mathcal{T}$ be a rank-decomposition tree of G. Let $\{X, Y\}$ be a partition of V separated by an edge e of $\mathcal{T}$. Without loss of generality, we assume that $|X| \geq 3$ and $|Y| \geq 3$. Now we consider the set X. We refer a *component* of $G[X]$ as a maximal connected subgraph.

Lemma 1. *If $G[X]$ contains a component S of size at least 3, then the rankwidth of G is at least 3.*

Proof. We have the following three cases:

1. S contains only outer vertices.
2. S contains only inner vertices.
3. S contains both outer and inner vertices.

We first consider Case 1. Since S contains only outer vertices, $G[S]$ is a path like Figure 4.

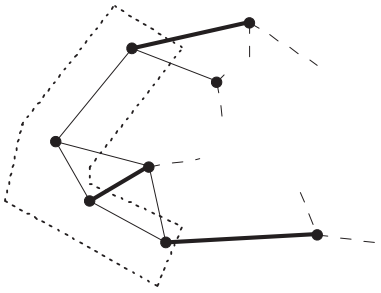


Figure 4: S contains only outer vertices.

Thus we can find three edges (bold edges in Figure 4) such that the rankwidth of G is at least 3.

Figure 5 shows Case 2. It is easy to find that there are three edges (bold edges in Figure 5) such that the rankwidth of G is at least 3.

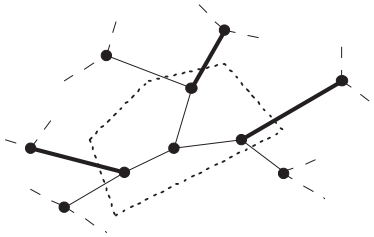


Figure 5: S contains only inner vertices.

Similarly, Case 3 can be proved in a similar way. Figure 6 shows a possible partition, *e.g.*, there are two outer vertices in S.

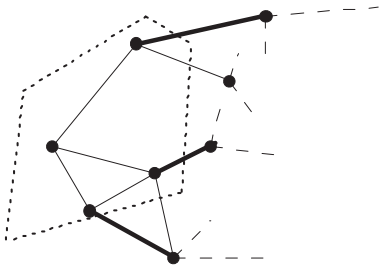


Figure 6: S contains both outer and inner vertices.

For the subcase that S contains only one outer vertex can be shown in a similar argument. Therefore, this lemma holds. □

Lemma 2. *If $G[X]$ contains at least three components, then the rankwidth of G is at least 3.*

Proof. The simplest way to show this lemma is to find three independent edges in G such that $rwd(G)$ is at least 3. The left graph depicted in Figure 7 shows the smallest graph satisfying the condition. It is easy to check that its rankwidth is 3. For the other possibilities, the distance between any two components is at least two, *e.g.*, the graph depicted in the right of Figure 7. Obviously, we can find three edges (*e.g.*, bold edges in Figure 7) such that its rankwidth is at least 3. Therefore, our lemma holds. □

Lemma 3. *If $G[X]$ contains exactly two components and there is no component containing at least three vertices, then the rankwidth of G is at least 3.*

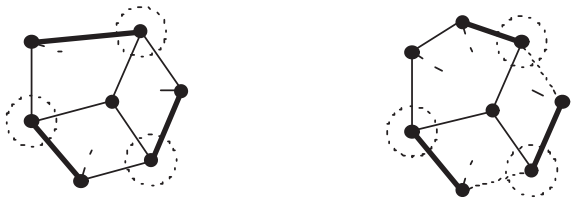


Figure 7: Possible cases for three components of $G[X]$.

Proof. Since $|X| \geq 3$, at least one component contains two vertices. Assume that this component is $S = \{a_1, a_2\}$. We have the following cases:

1. $|N(a_1) \cap N(a_2)| = 0$.
2. $|N(a_1) \cap N(a_2)| = 1$.
3. $|N(a_1) \cap N(a_2)| = 2$.

Note that since G is Halin, it is impossible that $|N(a_1) \cap N(a_2)| > 2$.

In Case 1, since $N(a_1) \cap N(a_2) = \varnothing$, it is easy to check that there exist three edges such that $rwd(G) \geq 3$. Figure 8 shows the case.

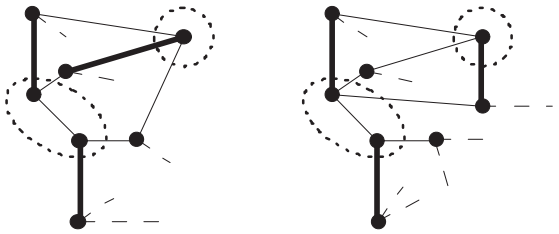


Figure 8: The Case 1.

For Case 2, we assume that b_1 is the common neighbor of a_1 and a_2. Then $\{a_1, a_2, b_1\}$ forms a triangle. Thus at least one of a_1 and a_2 is an outer vertex. Figure 9 shows that no matter which case $rwd(G) \geq 3$.

Now we consider the last case that $|N(a_1) \cap N(a_2)| = 2$. Since G is Halin, these two components cannot be too close. Figure 10 shows this case.

Since these two components are not too close, vertex b_x must exist. Thus the rankwidth of G

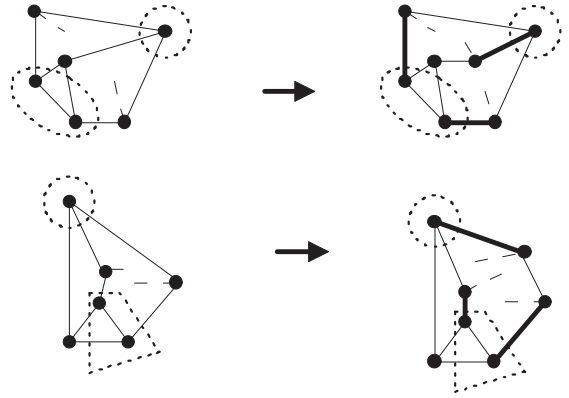


Figure 9: The Case 2.

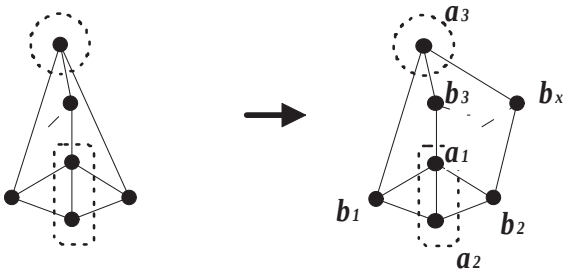


Figure 10: The Case 3.

can be computed as follows.

$$rwd(G) = \begin{pmatrix} & a_1 & a_2 & a_3 & \cdots & a_{|X|} \\ b_1 & 1 & 1 & * & \cdots & * \\ b_2 & 1 & 1 & * & \cdots & * \\ b_3 & 1 & 0 & * & \cdots & * \\ b_x & 0 & 0 & 1 & & \\ & * & * & * & & \\ \vdots & \vdots & \vdots & \vdots & & \vdots \\ b_{|Y|} & * & * & * & \cdots & * \end{pmatrix} \geq 3$$

Therefore, our lemma holds. □

2.2 The rank-decomposition trees of Halin graphs

According to the discussion of previous section, only few Halin graphs have rankwidth at most 2. In this section, we propose an algorithm to construct a rank-decomposition tree in rankwidth 3 for a Halin graph with rankwidth at least 3. Due to previous results, our algorithm is optimal.

For a Halin graph $G = (V, E)$, let $G = T + C$. Note that C is the outer cycle of G and vertices in C are leaves of T. Our algorithm works from C in counterclockwise order. To T, we works from leaves to center. For a vertex v, Function $Make\_Tree(v)$ construct a tree containing only one vertex v and this tree is labeled with v. For two vertices u and v, Function $Join\_Trees(u, v)$ constructs a tree by creating a new node and linking the two trees with labels u and v. The resulting tree is labeled with v. The detail of our algorithm is as follows.

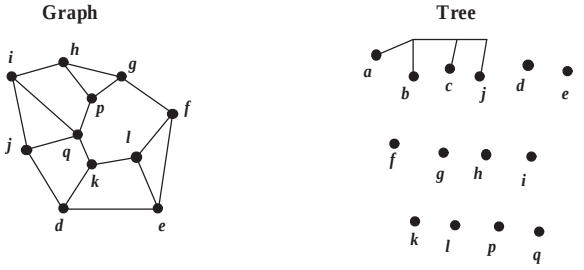
Algorithm 1 buildtree(a Halin graph $G = (V, E) = T + C$)

let $V = \{v_1, \ldots, v_n\}$ and $Q = \varnothing$ be an empty queue;
let $C = (c_1, \ldots, c_p)$ be the outer cycle of G in counterclockwise order;
for $i = 1$ to n **do**
  $number(v_i) = degree(v_i)$;
  $Make\_Tree(v_i)$;
end for
for $i = 1$ to p **do**
  let v be the neighbor of c_i in T;
  $number(v) = number(v) - 1$;
  if $number(v) == 1$ **then**
    $Enqueue(Q, v)$;
  end if
end for
for $Q \neq \varnothing$ **do**
  $v = Dequeue(Q)$;
  if $N(v) \setminus C \neq \varnothing$ **then**
    let u be the vertex in $N(v) \setminus C$;
    $number(u) = number(u) - 1$;
    if $number(u) == 1$ **then**
      $Enqueue(Q, u)$;
    end if
  end if
  let $X(= \{x_1, \ldots, x_k\}) = N(v) \cap C$ in counterclockwise order;
  for $i = 1$ to $k - 1$ **do**
    $Join\_Trees(x_i, x_{i+1})$;
  end for
  $Join\_Trees(x_k, v)$;
  replace X with v in C;
end for
return the resulting tree $\mathcal{T}$;

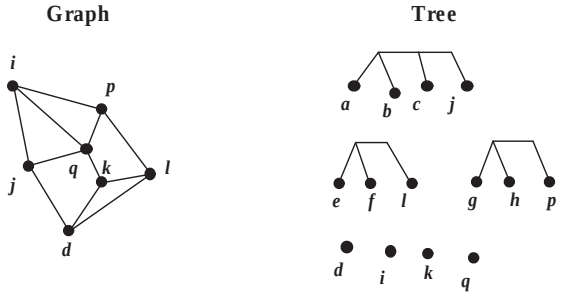
In the following, we show an example for constructing a rank-decomposition tree step by step. At first, the outer cycle of G is $C = (a, b, c, d, e, f, g, h, i)$. Our decomposition tree $\mathcal{T}$ is a forest containing n single trees.

After scanning the outer vertices, we obtain that $Q = (j, l, p)$. We then process the vertex j.

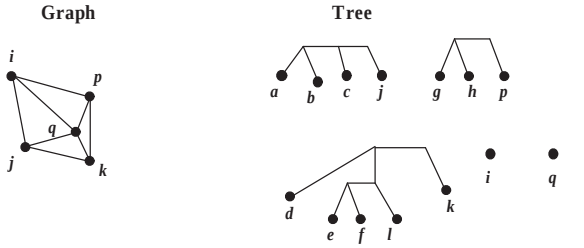
The resulting $\mathcal{T}$ is as the following figure and C becomes (j, d, e, f, g, h, i).



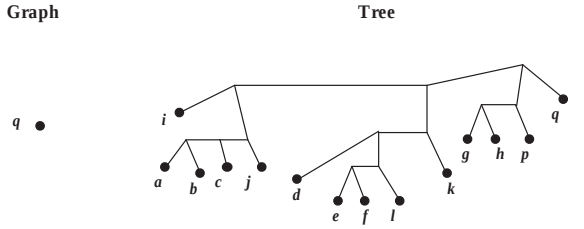
We then process the vertices l and p. After this process, Q contains only one vertex k and C becomes to (j, d, l, p, i). The following figure shows the results.



Similarly, after processing vertex k, we have that $Q = (q)$ and $C = (j, k, p, i)$. The following figure shows the results.



Finally, after processing vertex q, $Q = \varnothing$, $C = (q)$, and we obtain the rank-decomposition tree $\mathcal{T}$ shown as follows.



It is easy to check that our algorithm runs in $O(n + m)$ time. Since Halin graphs are planar, $m = O(n)$. Thus, our algorithm runs in $O(n)$ time. For this version, we omit the proof that the width of $\mathcal{T}$ is 3. We conclude this section in the following theorem.

Theorem 1. *The rankwidth of the class of Halin graphs is at most 3. Further, an optimal rank-decomposition tree of a Halin graph can be obtained in linear time.*

3 Conclusion

In this paper we show that the rankwidth of a Halin graph is at most three, and construct a corresponding rank-decomposition tree in linear time. It is interesting to construct optimal rank-decomposition trees for other classes of graphs.

References

[1] H.J. Bandelt and H.M. Mulder, Distance-hereditary graphs, *Journal of Combinatorial Theory Series B* **41** (1986) 182–208.

[2] B. Courcelle and S. Olariu, Upper bounds to the clique width of graphs, *Discrete Applied Mathematics* **101** (2000) 77–114.

[3] R. Diestel, *Graph Theory*, Springer, 2000.

[4] M.R. Fellows, F.A. Rosamond, U. Rotics, S. Szeider, Proving NP-hardness for clique-width II: Non-approximability of clique-width, Report TR05-081, Revision 01, Electronic Colloquium on Computational Complexity, 2005.

[5] M.R. Fellows, F.A. Rosamond, U. Rotics, S. Szeider, Clique-width minimization is NP-hard, in: *Proceedings of the 38th ACM Symposium on Theory of Computing*, 354–362, 2006.

[6] P. Hlinn, S. Oum, D. Seese, and G. Gottlob, Width Parameters Beyond Tree-width and Their Applications, *The Computer Journal* **51** (2008) 326–362.

[7] S. Oum, Rank-width and vertex-minors, *Journal of Combinatorial Theory, Series B* **95** (2005) 79–100.

[8] S. Oum, Approximating rank-width and clique-width quickly, *Proceedings of 31st International Workshop on Graph-Theoretic Concepts in Computer Science*, LNCS, **3787** (2005) 49–58.

[9] S. Oum and P. Seymour, Approximating clique-width and branch-width, *Journal of Combinatorial Theory, Series B* **96** (2006) 514–528.

[10] S.-L. Peng, G.-D. Chen, H.-Y. Tsou, I.-T. Li, and Y.-F. Li, Constructing optimal rank-decomposition trees of biconnected outerplanar graphs, *Proceedings of the 25th Workshop on Combinatorial Mathematics and Computation Theory*, 424-428, 2008.

[11] N. Robertson and P.D. Seymour, Graph minors I: Excluding a forest, *Journal of Combinatorial Theory, Series B* **35** (1983) 39–61.

[12] N. Robertson and P. Seymour, Graph minors II: Algorithmic aspects of tree-width, *Journal of algorithms* **7** (1986) 309–322.

[13] N. Robertson and P. Seymour, Graph minors X: Obstructions to tree- decomposition, *Journal of Combinatorial Theory, Series B* , **52** (1991) 153–190.

[14] R. Halin. Studies on minimally n-connected graphs. *Combinatorial Mathematics and its applications*,D.J.A. Welsh,ed., Academic Press, New York(1971) 129–136.